

Group #31, Hovik Grigoryan (hg483), Ethan Kim (ejk252), Dain Lee (ld499)

Robot Design and Strategy Overview:

From the mechanical perspective, our robot has large wheels which allows it to store cubes underneath, while the back side is blocked with cardboard to avoid losing the cubes from behind. The robot has two fixed arms in front of it to guide the cubes underneath. (see Figure 1 and appendix C figure 3) The robot also has a cardboard box attached to its left side in order to collect and store even more cubes while driving forward. The color sensor is near the front of the robot, close to the floor in order to accurately detect the color.

The essential electronics that make up the robot are two motors for the wheels controlled by 2 h-bridges, as well as a color sensor. The color sensor and the arduino are powered by a 9V battery, while the h-bridges and the motors are powered by a 6V battery pack. The robot stores the arduino board on the lower level, and the breadboard with h-bridges is wired on top.

The strategy involves the following steps: The robot drives forward. When it detects a color change, which means that it is in the middle section of the board, it turns right by 45 degrees and continues driving forward. When it detects a black border, it drives back slightly, then turns right by 180 degrees and drives forward. The robot only turns right, which ensures that the cubes stored within the cardboard box stay within the box (if the robot turned left, some cubes would be lost from inside the box). For both the half-field change detection and black border detection, we only used a color sensor and it proved to be sufficiently reliable. When by itself, the robot never drives off the field.



Figure 1: Team #31's robot on the competition day (final design)

Design Process Reflection:

Our initial plan for the robot included four QTI sensors, a color sensor and four motors to extract more torque when moving. We quickly found out that purchasing two additional motors would be tight for our budget, so we decided to stick with two instead. Additionally, the QTI sensors turned out to be very tricky, the two we were provided with were highly unreliable. So, after running tests we found out that the color sensor is sufficient enough to detect both color changes and borders. So, while working on the robot, our design became more simple, in terms of the number of sensors. We also added the cube-collection box at the end as we initially planned on only collecting cubes under the robot. We came to find out that the area underneath the robot was not large enough to collect many cubes. The cardboard box fixed the issue quickly. Initially we also added small extensions to elevate the backwheel of the robot. After running tests, we saw that this negatively impacted the stability of the robot, and it started tipping and falling easily, so we backtracked this idea. Throughout the project, our robot design altered significantly due to budget constraints and design feasibility, as well as feedback from tests.

Competition Analysis:

Our robot participated in 7 matches, out of which we lost 5, won 1 and had a draw in 1. The first loss came because of the uncharged arduino battery, and our robot did not move at all. We fixed the issue quickly by replacing the battery with a working one. In our other matches we were rather unlucky. We did not expect our robot to be so light compared to other robots, and it ended up getting pushed out of the board by the competitor's robot in almost all the matches. Our nuanced turn strategy was countered by brute-force pushing. At the beginning of each game our robot managed to collect many cubes, but it mostly lost them when getting pushed backward. The relative light weight of our robot is associated with the fact that the cube-collecting box was made with cardboard, and was not 3D printed, whereas other robots had larger 3D-printed parts.

Conclusions:

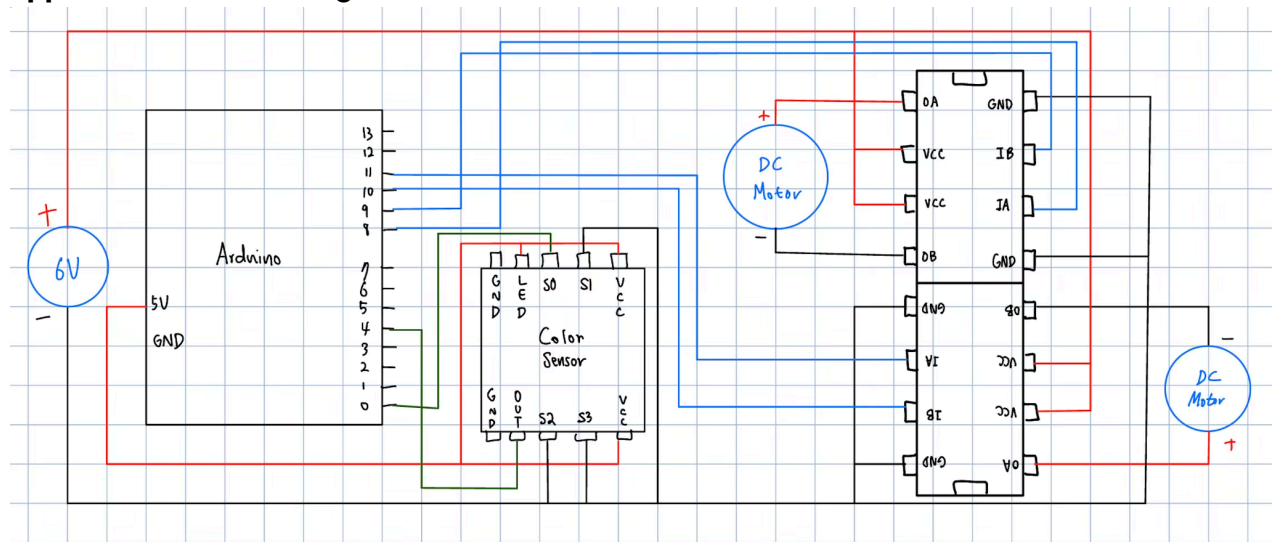
Something we would change is that we would make our robot heavier. One way to do this would be using the remainder of our budget to 3D print the cube box instead of using cardboard. We would generally just put more emphasis on the possibility of a collision with the opponent. A way to tackle this could have been having a potentiometer to detect the opponent robot ahead of ours, and perhaps avoid the opponent by having our robot either roll back or turn when detecting them. Other than that, our robot was able to stay on the board on its own and successfully collect many cubes before getting pushed off. We would recommend students who will take the course next year to not overcomplicate their robot by overloading it with too many sensors that are too hard to manage together. One QTI sensor, one color sensor and one potentiometer would be sufficient. Also, it is important to note that even though the milestones are passed individually, during the competition a collision with the opponent

is almost unavoidable, and a large emphasis must be placed on this and how your robot will handle head-to-head collisions.

Appendix A: Bill of Materials:

- Two 3d-printed back-wheel extensions, printed with PLA (see appendix C)
Weight of both together: 0.01g
Price of both together: **\$2.00**
- Two 3d-printed arms, printed with PLA (see appendix C)
Weight of both together: 1g
Price of both together: **\$2.40**
- Two large wheels ordered from Pololu (<https://www.pololu.com/product/4929>)
Price of both together: **\$15.50**

Appendix B: Circuit Diagram



Appendix C:

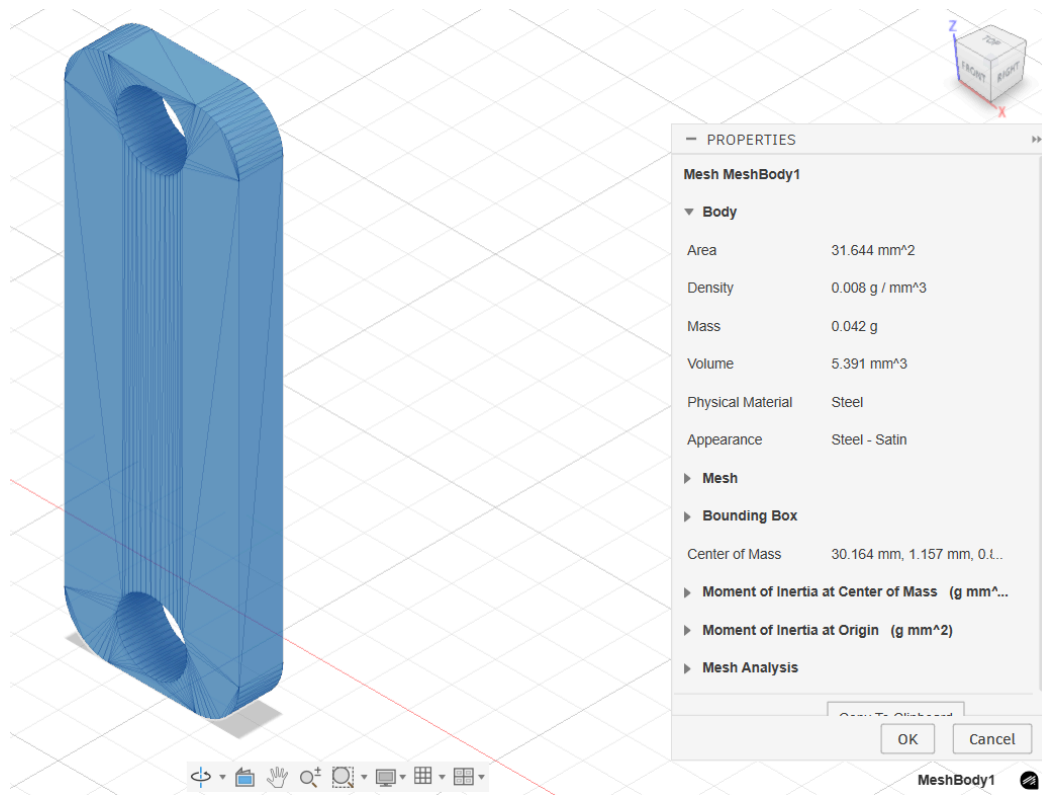


Figure 2: Sketch and volume of the back wheel extensions

The back wheel extensions have a volume of $5.391 \text{ mm}^3 = 0.005391 \text{ cm}^3$.
 We used PLA to print those, which have a density of $0.75 \times 1.25 = 0.938 \text{ g/cm}^3$.
 Therefore each of the two backwheel extensions has a weight of $0.938 \times 0.005391 = 0.005 \text{ g}$.
 The two of those together weigh about **0.01g**.

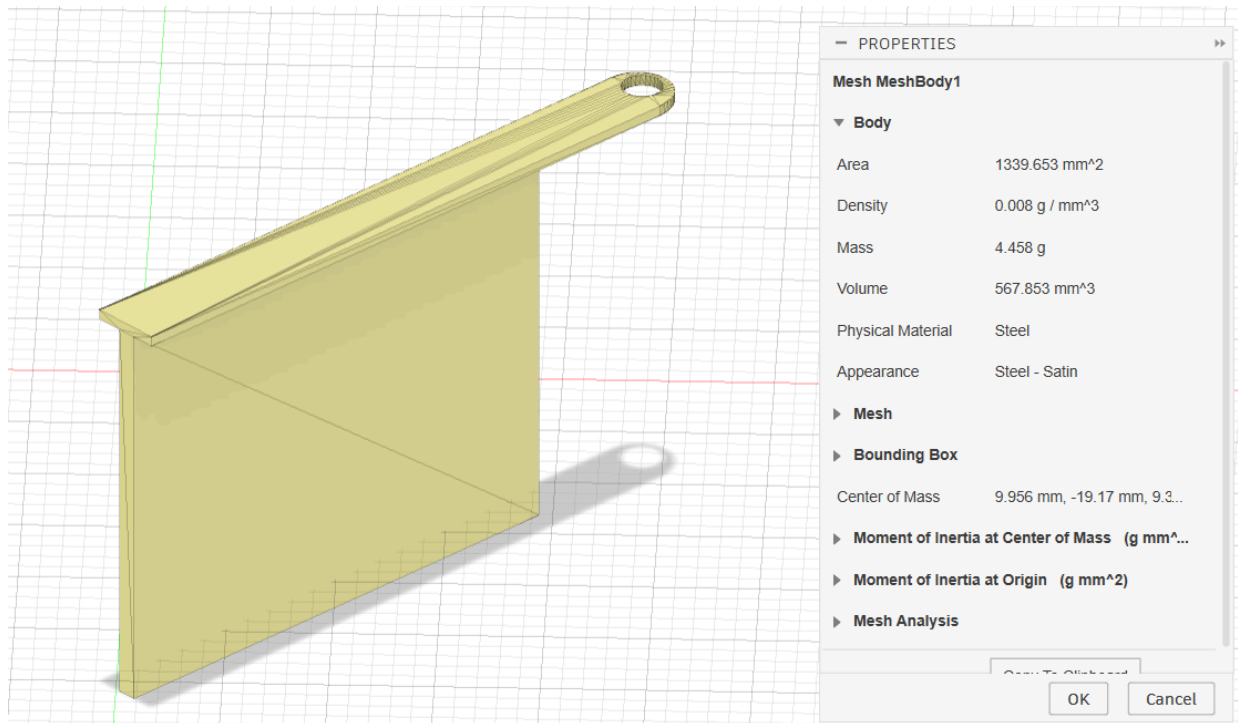
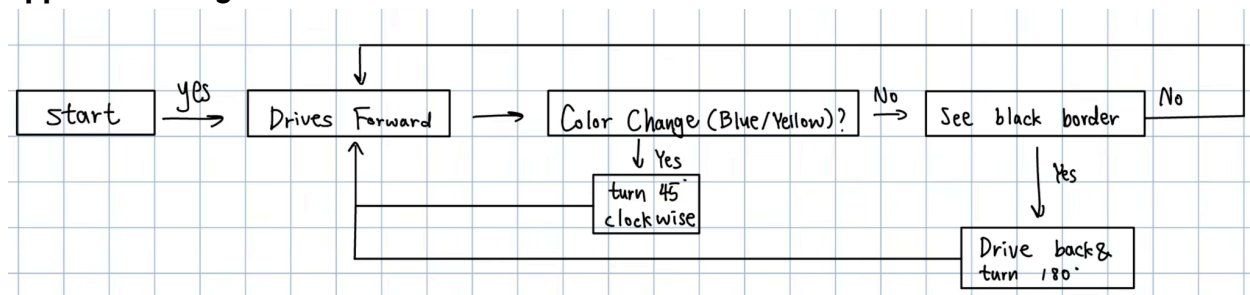


Figure 3: Sketch and volume of the 3d-printed arms

The back wheel extensions have a volume of $567.853 \text{ mm}^3 = 0.537853 \text{ cm}^3$.
 We used PLA to print those, which have a density of $0.75 \times 1.25 = 0.938 \text{ g/cm}^3$.
 Therefore each of the two backwheel extensions has a weight of $0.938 \times 0.537853 = 0.5$
 The two of those together weigh about **1g**.

Appendix D: Algorithm Flowchart



Appendix E:

```

#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>

// PWM duty cycle: 0-255. Lower = slower/weaker motors.
// Tune this so both wheels match the weaker motor.
#define MOTOR_DUTY 180

```

```

volatile uint8_t pwm_counter = 0;           // counts overflow cycles to
implement software PWM
volatile uint8_t motor_target = 0;         // desired PORTB state (set by
movement functions)

volatile int timer = 0;                    // stores captured timer value
from color sensor
int period = 0;                           // calculated period of color
sensor output signal in microseconds

ISR(TIMER0_OVF_vect) {                    // interrupt service routine
called on every Timer0 overflow
    pwm_counter++;                         // increment the software PWM
cycle counter
    if (pwm_counter < MOTOR_DUTY) {       // during the "high" phase of the
PWM cycle
        PORTB = motor_target;             // drive motors with the desired
port state
    } else {
        PORTB = 0b00000000;              // off during low phase
    }
}

void pwm_init(void) {
    TCCR0A = 0x00;                         // set Timer0 to normal mode
    TCCR0B = (1 << CS01);                  // clk/8 prescaler
    TIMSK0 = (1 << TOIE0);                 // enable overflow interrupt
    sei();                                  // enable global interrupts
}

void set_motors(uint8_t portb_state) {     // sets the motor target state to
be applied each PWM cycle
    motor_target = portb_state;            // update target so ISR picks it
up on next cycle
}

void drive_forward(void) {
    set_motors(0b00000101);               // energize both motors in the
forward direction

```

```

}

void drive_forward_1_foot(void) {
    set_motors(0b00001010);           // start motors moving forward
    _delay_ms(1778);                   // delay calibrated to travel
    approximately 1 foot
    set_motors(0b00000000);           // stop motors
}

void drive_forward_6_inches(void) {
    set_motors(0b00001010);           // start motors moving forward
    _delay_ms(889);                   // delay calibrated to travel
    approximately 6 inches
    set_motors(0b00000000);           // stop motors
}

void drive_backward_1point5_feet(void) {
    set_motors(0b00000101);           // start motors in reverse
    direction
    _delay_ms(2667);                   // delay calibrated to travel
    approximately 1.5 feet
    set_motors(0b00000000);           // stop motors
}

void drive_backward(void) {
    set_motors(0b00001010);           // start motors in reverse
    direction
    _delay_ms(400);                   // run in reverse for 400ms
    set_motors(0b00000000);           // stop motors
}

void turn_left(void) {
    set_motors(0b00001001);           // spin motors in opposite
    directions to turn left
    _delay_ms(480);                   // delay calibrated to turn
    approximately 90 degrees left
    set_motors(0b00000000);           // stop motors
}

void turn_left45(void) {

```

```

        set_motors(0b00001001);           // spin motors in opposite
directions to turn left
        _delay_ms(240);                     // delay calibrated to turn
approximately 45 degrees left
        set_motors(0b00000000);           // stop motors
    }

void turn_left_180(void) {
        set_motors(0b00001001);           // spin motors in opposite
directions to turn left
        _delay_ms(750);                     // delay calibrated to turn
approximately 180 degrees left
        set_motors(0b00000000);           // stop motors
    }

void turn45(void){
        set_motors(0b00001001);           // spin motors in opposite
directions to turn left
        _delay_ms(240);                     // delay calibrated to turn
approximately 45 degrees
        set_motors(0b00000000);           // stop motors
    }

void turn_right(void) {
        set_motors(0b00000110);           // spin motors in opposite
directions to turn right
        _delay_ms(500);                     // delay calibrated to turn
approximately 90 degrees right
        set_motors(0b00000000);           // stop motors
    }

void turn_right45(void) {
        set_motors(0b00000110);           // spin motors in opposite
directions to turn right
        _delay_ms(240);                     // delay calibrated to turn
approximately 45 degrees right
        set_motors(0b00000000);           // stop motors
    }

void turn_right_180(void) {

```



```

        set_motors(0b00000110);           // spin motors in opposite
directions to turn right
        _delay_ms(750);                     // delay calibrated to turn
approximately 180 degrees right
        set_motors(0b00000000);           // stop motors
    }

void random_turn(){
    int rand = random(2);                   // generate a random number: 0 or
1
    if(rand == 0){
        turn_right45();                     // turn right 45 degrees if rand
is 0
    }
    else{
        turn_left45();                     // turn left 45 degrees if rand is
1
    }
}

void stop(void){
    set_motors(0b00000000);               // set all motor pins low to halt
movement
}

ISR(PCINT2_vect) {
    if (PIND & 0b00010000) {               // if PD4 just went HIGH (rising
edge of sensor pulse)...
        TCNT1 = 0;                         // reset Timer1 to start measuring
pulse width
    } else {
        timer = TCNT1;                     // falling edge: capture elapsed
Timer1 ticks as pulse width
    }
}

void initColor() {
    DDRD  = 0b00000001;                   // PD0=S0 out, PD5=LED out
    PORTD = 0b00000001;                   // S0 HIGH

```

```

    PCICR = 0b00000100;           // enable pin-change interrupts on
PORTD (PCIE2)
    PCMSK2 = 0b00000000;           // disable all PORTD pin-change
interrupt pins initially
    sei();                           // enable global interrupts
    TCCR1A = 0b00000000;           // set Timer1 to normal mode (no
PWM output)
    TCCR1B = 0b00000001;           // start Timer1 with no prescaler
}

int getColor() {
    PCMSK2 = 0b00010000;           // arm PD4
    _delay_ms(10);                 // wait 10ms to capture at least
one full sensor pulse cycle
    PCMSK2 = 0b00000000;           // disarm

    cli();                           // disable interrupts to safely
read the volatile timer value
    int snapshot = timer;           // copy the captured pulse width to
a local variable
    sei();                           // re-enable global interrupts

    period = (2 * snapshot) / 16;    // convert Timer1 ticks to
microseconds (2× for full period, /16 for 16MHz)
    return period;                 // return the calculated period in
microseconds
}

int main(void) {
    Serial.begin(9600);             // initialize UART serial at 9600
baud for debug output
    initColor();                   // configure color sensor and
timers
    int previousPeriod = getColor(); // record the initial color reading
as the baseline
    DDRB = 0b00001111;             // set PB0-PB3 as outputs to drive
the motor H-bridge
    pwm_init();                     // configure Timer0 and enable
software PWM interrupt

```

```

while (1) {
    int measuredPeriod = getColor();    // take a fresh color sensor
reading each loop iteration
    drive_forward();                    // continuously drive forward
between sensor checks
    if ((abs(measuredPeriod - previousPeriod) > 90) & (measuredPeriod
< 245)) { //change second if value depending on day (it is to not
interfere with black measurement)
        Serial.println("Color changed!");    // print color change
detection
        turn_right45();                        // steer right 45 degrees
to follow the color boundary
        drive_forward();                        // resume forward motion
after the turn
        previousPeriod = measuredPeriod;    // update baseline to new
color
        _delay_ms(500);                        // wait 500ms to let the
robot settle before next read
        previousPeriod = measuredPeriod;    // update baseline to new
color
    }
    if((measuredPeriod > 350) & ( measuredPeriod < 500)) { //sets
range for black period
        Serial.println("Black measured");    // print black-line
detection
        drive_backward();                        // back away from the
black line
        turn_right_180();                        // spin 180 degrees to
reverse heading
        drive_forward();                        // drive away from the
black boundary
        _delay_ms(500);                        // wait 500ms before
resuming normal sensing
    }

    Serial.println(measuredPeriod);    // print current sensor period
value for live monitoring
    _delay_ms(100);                        // short pause to avoid
flooding the serial output
}

```

```
    return 0;  
}
```